



TECNOLÓGICO
NACIONAL DE MÉXICO®



INGENIERIA EN TECNOLOGIAS DE LA INFORMACIÓN Y COMUNICACIONES

PROGRAMACIÓN WEB AVANZADA

ACTIVIDAD 3:

API RESTFUL CRUD CON EXPRESS

PROFESORA:

ALMA ROSA SANCHEZ AGUIAR

ALUMNA:

YOSELIN DE JESUS SOLIS CERVANTES

NÚMERO DE CONTROL:

221140013

FECHA:

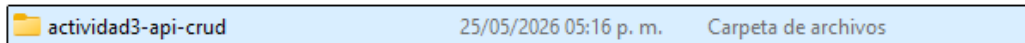
26/05/2026

Introducción

En esta actividad se desarrolló una API RESTful utilizando Node.js y Express, implementando operaciones CRUD para la gestión de tareas. Se utilizó una estructura organizada mediante rutas, controladores y middlewares, además del manejo de respuestas HTTP y validaciones utilizando Postman para las pruebas de funcionamiento.

El proyecto fue desarrollado siguiendo una arquitectura organizada por responsabilidades, permitiendo separar la lógica de negocio, las rutas y las validaciones en diferentes carpetas del sistema.

Primero se creó una carpeta en donde se va a guardar todo el proyecto



Inicializar Node.js

Se inicializó el proyecto utilizando Node.js mediante el comando `npm init -y`, lo que permitió crear el archivo `package.json` con la configuración básica del proyecto.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\actividad3-api-crud>
PS C:\actividad3-api-crud> npm init -y

Wrote to C:\actividad3-api-crud\package.json:

{
  "name": "actividad3-api-crud",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Instalar dependencias

Se instalaron las dependencias necesarias para el desarrollo del proyecto, incluyendo Express para la creación del servidor y Nodemon para facilitar la ejecución automática durante los cambios en el código.

```
npm install expresstividad3-api-crud>

added 66 packages, and audited 67 packages in 6s

24 packages are looking for funding
  run `npm fund` for details

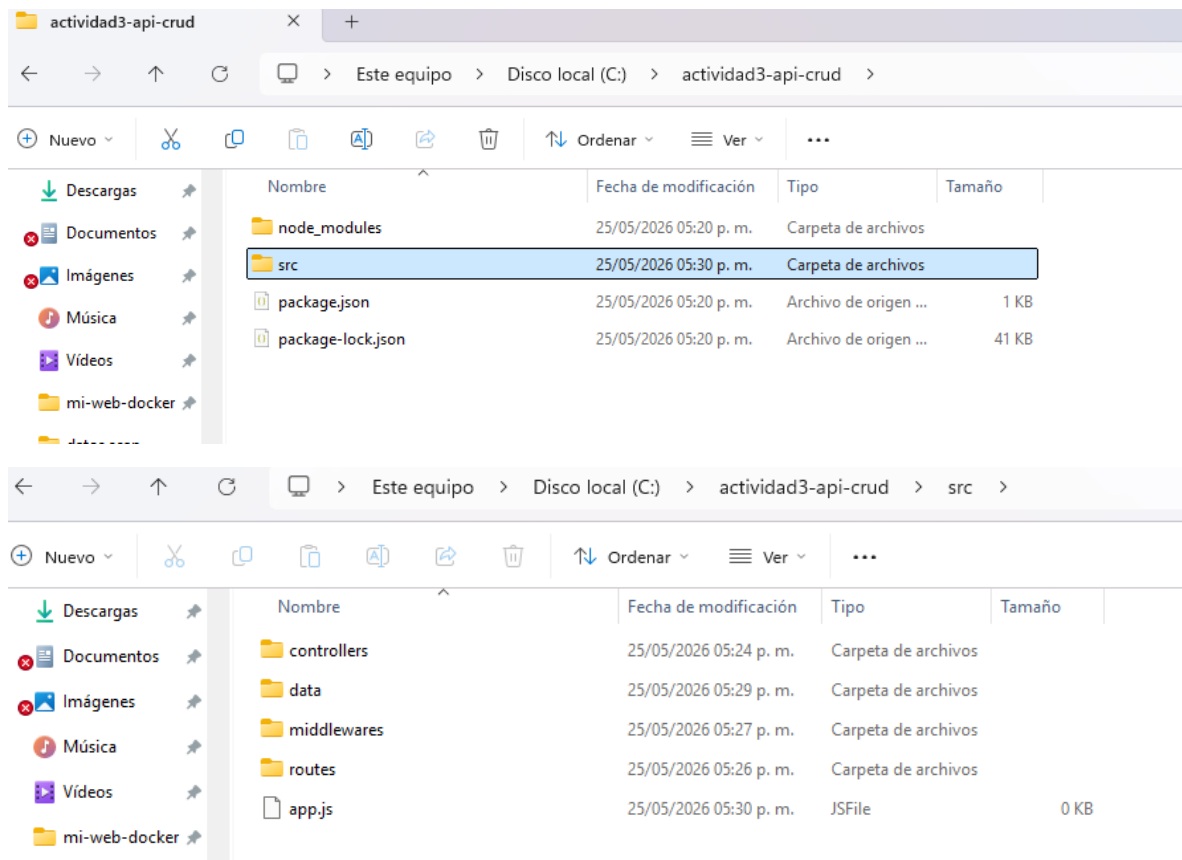
found 0 vulnerabilities
PS C:\actividad3-api-crud> npm install --save-dev nodemon

added 26 packages, and audited 93 packages in 4s

29 packages are looking for funding
  run `npm fund` for details

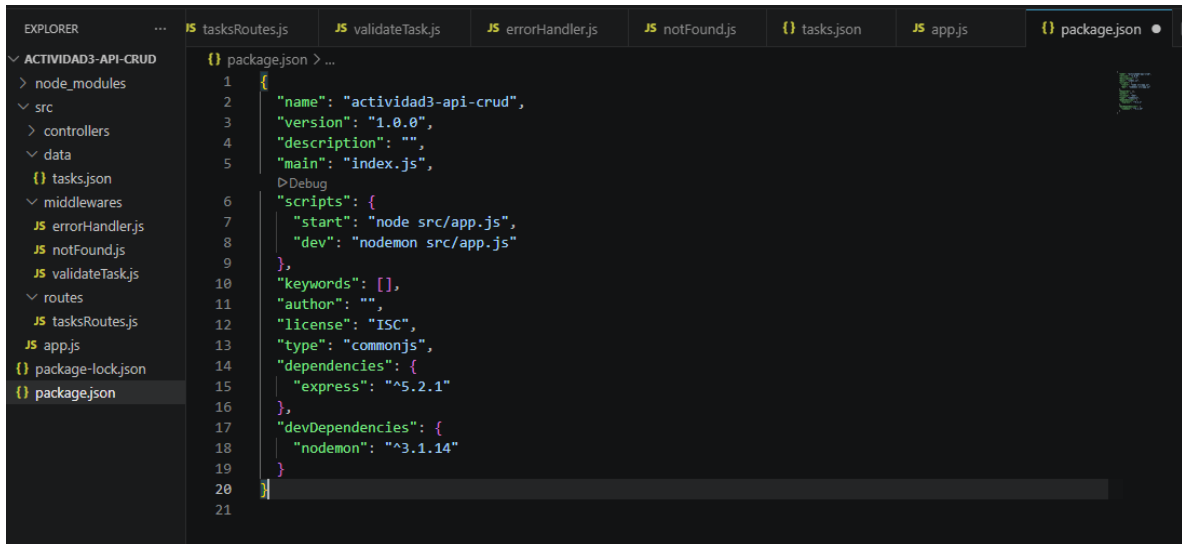
found 0 vulnerabilities
```

Crear estructura del proyecto



Configurar package.json

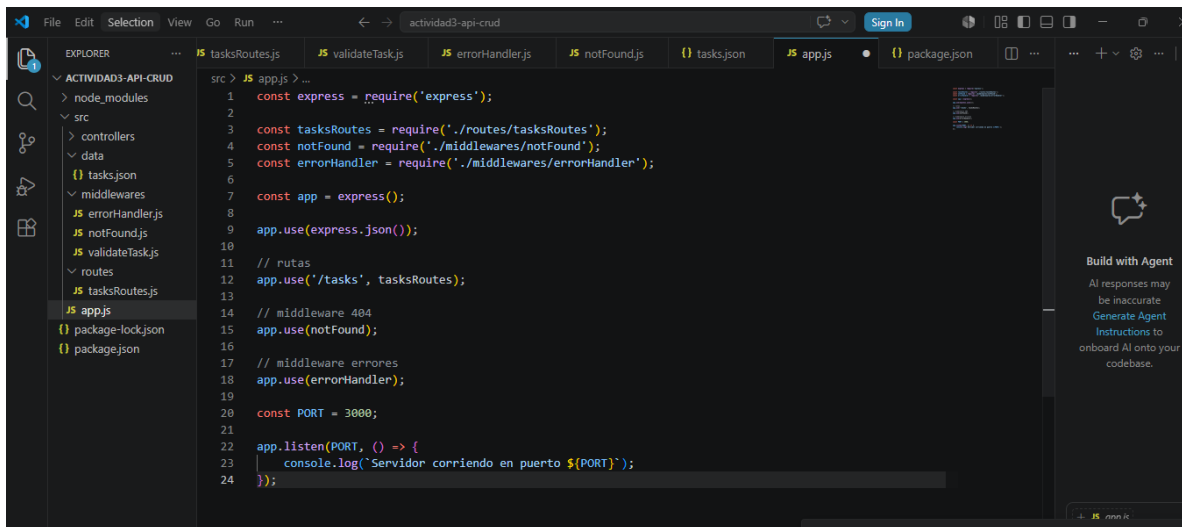
Se configuró el archivo package.json agregando scripts de ejecución para iniciar el servidor utilizando los comandos npm start y npm run dev.



```
1 {
2   "name": "actividad3-api-crud",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "start": "node src/app.js",
8     "dev": "nodemon src/app.js"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13  "type": "commonjs",
14  "dependencies": {
15    "express": "^5.2.1"
16  },
17  "devDependencies": {
18    "nodemon": "^3.1.14"
19  }
20 }
```

Crear archivo app.js

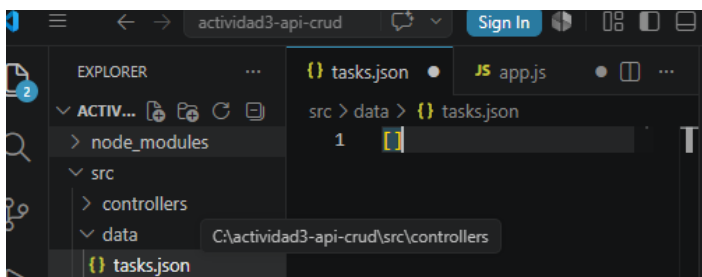
Se creó el archivo app.js como punto principal de la aplicación, donde se configuró Express, las rutas del sistema y los middlewares para el manejo de errores.



```
1 const express = require('express');
2
3 const tasksRoutes = require('./routes/tasksRoutes');
4 const notFound = require('./middlewares/notFound');
5 const errorHandler = require('./middlewares/errorHandler');
6
7 const app = express();
8
9 app.use(express.json());
10
11 // rutas
12 app.use('/tasks', tasksRoutes);
13
14 // middleware 404
15 app.use(notFound);
16
17 // middleware errores
18 app.use(errorHandler);
19
20 const PORT = 3000;
21
22 app.listen(PORT, () => {
23   console.log('Servidor corriendo en puerto ${PORT}');
24 });
```

Crear archivo JSON

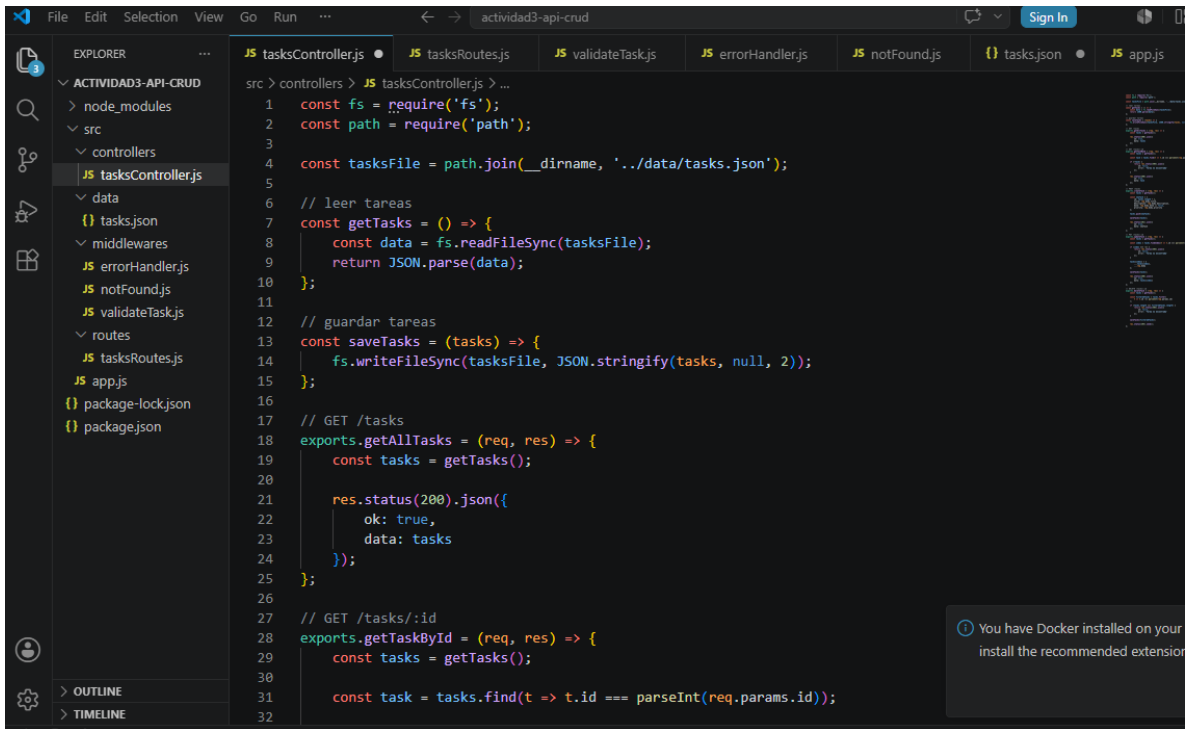
Se creó un archivo tasks.json para almacenar la información de las tareas de manera local sin utilizar una base de datos.



```
1 [ ]
```

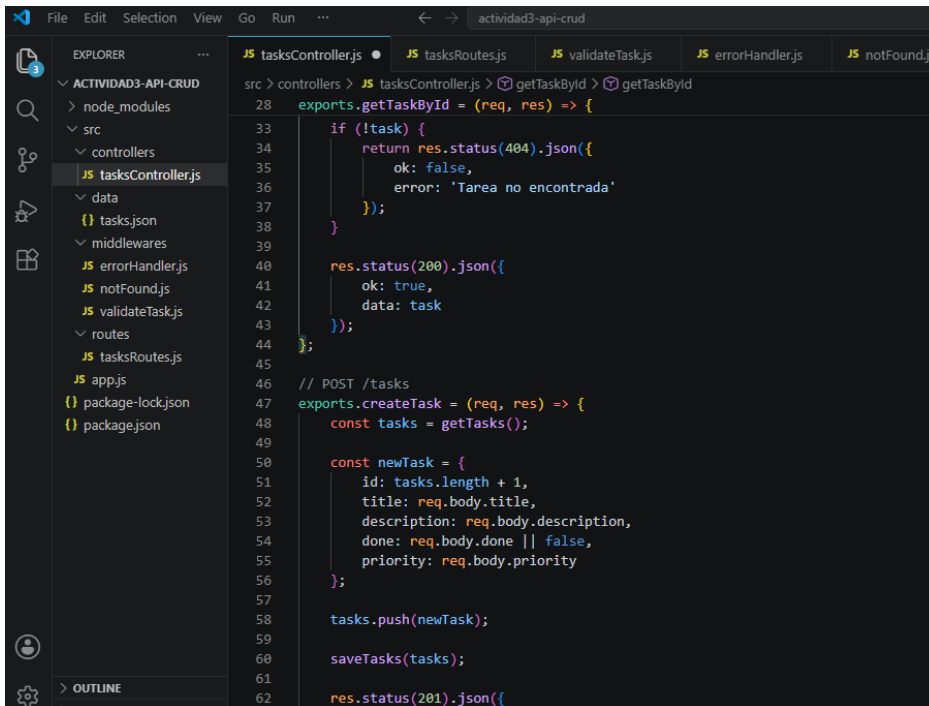
Crear controlador

Se implementó un controlador encargado de manejar la lógica de las operaciones CRUD, permitiendo crear, consultar, actualizar y eliminar tareas.



This screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `tasksController.js` file. The code implements the `getTasks` function, which reads the `tasks.json` file and returns the tasks as a JSON array. It also includes a `getAllTasks` export and a `getTaskById` export that uses `getTasks` to find a specific task.

```
1  const fs = require('fs');
2  const path = require('path');
3
4  const tasksFile = path.join(__dirname, '../data/tasks.json');
5
6  // leer tareas
7  const getTasks = () => {
8    const data = fs.readFileSync(tasksFile);
9    return JSON.parse(data);
10 };
11
12 // guardar tareas
13 const saveTasks = (tasks) => {
14   fs.writeFileSync(tasksFile, JSON.stringify(tasks, null, 2));
15 };
16
17 // GET /tasks
18 exports.getAllTasks = (req, res) => {
19   const tasks = getTasks();
20
21   res.status(200).json({
22     ok: true,
23     data: tasks
24   });
25 };
26
27 // GET /tasks/:id
28 exports.getTaskById = (req, res) => {
29   const tasks = getTasks();
30
31   const task = tasks.find(t => t.id === parseInt(req.params.id));
32 }
```



This screenshot shows the Visual Studio Code editor with the file explorer on the left. The main editor window shows the `tasksController.js` file, specifically the `createTask` function. This function takes a request and response object, gets the current tasks, creates a new task object with a unique ID, title, description, done status, and priority, pushes it to the tasks array, saves the updated array to the `tasks.json` file, and returns a 201 status code.

```
28 exports.getTaskById = (req, res) => {
29   if (!task) {
30     return res.status(404).json({
31       ok: false,
32       error: 'Tarea no encontrada'
33     });
34   }
35
36   res.status(200).json({
37     ok: true,
38     data: task
39   });
40 };
41
42 // POST /tasks
43 exports.createTask = (req, res) => {
44   const tasks = getTasks();
45
46   const newTask = {
47     id: tasks.length + 1,
48     title: req.body.title,
49     description: req.body.description,
50     done: req.body.done || false,
51     priority: req.body.priority
52   };
53
54   tasks.push(newTask);
55
56   saveTasks(tasks);
57
58   res.status(201).json({
59     ok: true,
60     data: newTask
61   });
62 }
```

```

47 exports.createTask = (req, res) => {
63     ok: true,
64     data: newTask
65   });
66 };
67
68 // PUT /tasks/:id
69 exports.updateTask = (req, res) => {
70   const tasks = getTasks();
71
72   const index = tasks.findIndex(t => t.id === parseInt(req.params.id));
73
74   if (index === -1) {
75     return res.status(404).json({
76       ok: false,
77       error: 'Tarea no encontrada'
78     });
79   }
80
81   tasks[index] = {
82     ...tasks[index],
83     ...req.body
84   };
85
86   saveTasks(tasks);
87
88   res.status(200).json({
89     ok: true,
90     data: tasks[index]
91   });
92 };
93

```

```

94 // DELETE /tasks/:id
95 exports.deleteTask = (req, res) => {
96   const tasks = getTasks();
97
98   const filteredTasks = tasks.filter(
99     t => t.id !== parseInt(req.params.id)
100   );
101
102   if (tasks.length === filteredTasks.length) {
103     return res.status(404).json({
104       ok: false,
105       error: 'Tarea no encontrada'
106     });
107   }
108
109   saveTasks(filteredTasks);
110
111   res.status(204).send();
112 };

```

Crear rutas

Se configuraron las rutas de la API RESTful para conectar cada endpoint con las funciones correspondientes del controlador.

```
const express = require('express');

const router = express.Router();

const {
  getAllTasks,
  getTaskById,
  createTask,
  updateTask,
  deleteTask
} = require('../controllers/tasksController');

const validateTask = require('../middlewares/validateTask');

router.get('/', getAllTasks);

router.get('/:id', getTaskById);

router.post('/', validateTask, createTask);

router.put('/:id', validateTask, updateTask);

router.delete('/:id', deleteTask);

module.exports = router;
```

Middleware de validación

Se implementó un middleware de validación para comprobar que los datos enviados por el usuario cumplieran con la estructura y tipos requeridos antes de procesar las peticiones.

```
JS tasksRoutes.js JS validateTask.js JS errorHandler.js JS notFound.js {} tasks.json
src > middlewares > JS validateTask.js > ...
1  const validateTask = (req, res, next) => {
2
3      const { title, done } = req.body;
4
5      if (!title || typeof title !== 'string') {
6          return res.status(400).json({
7              ok: false,
8              error: 'El campo title es obligatorio y debe ser texto'
9          });
10     }
11
12     if (done !== undefined && typeof done !== 'boolean') {
13         return res.status(400).json({
14             ok: false,
15             error: 'El campo done debe ser boolean'
16         });
17     }
18
19     next();
20 };
21
22 module.exports = validateTask;
```

Middleware error 500

Se implementó un middleware de manejo de errores internos del servidor para responder de manera consistente utilizando código HTTP 500.

```
JS tasksRoutes.js JS validateTask.js JS errorHandler.js JS notFound.js {} tasks.json
src > middlewares > JS errorHandler.js > ...
1  const errorHandler = (err, req, res, next) => {
2
3      console.error(err);
4
5      res.status(500).json({
6          ok: false,
7          error: 'Error interno del servidor'
8      });
9  };
10
11 module.exports = errorHandler;
```

Ejecutar proyecto

```
PS C:\actividad3-api-crud> npm run dev

> actividad3-api-crud@1.0.0 dev
> nodemon src/app.js

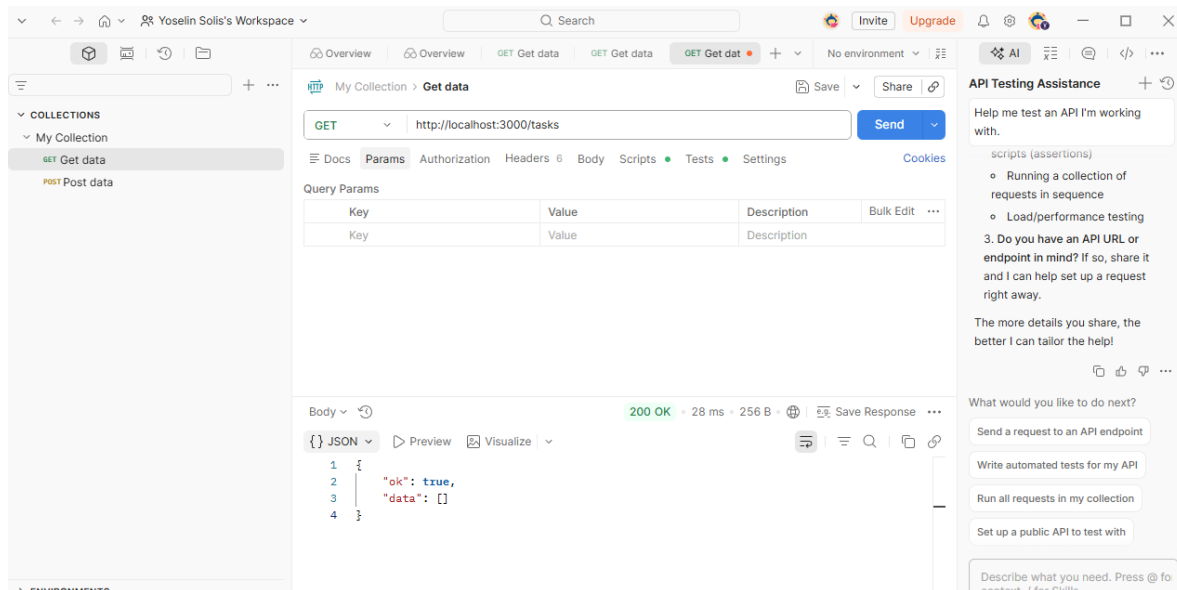
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/app.js`
[nodemon] clean exit - waiting for changes before restart
```

Probar endpoints en Postman

Se realizaron pruebas de funcionamiento utilizando Postman para verificar el correcto comportamiento de cada endpoint CRUD y el manejo de errores de la API.

Prueba del método GET para listar tareas

Se realizó una petición GET al endpoint /tasks para obtener la lista de tareas almacenadas en la API RESTful. La respuesta fue exitosa con código HTTP 200.



Método: POST para crear una tarea

Se utilizó el método POST para registrar una nueva tarea enviando datos en formato JSON. La API respondió correctamente con código HTTP 201 indicando que el recurso fue creado exitosamente.

POST http://localhost:3000/tasks

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings

Cookies

raw JSON

Schema Beautify

```
1 {
2   "title": "Hacer actividad CRUD",
3   "description": "Practicar Express",
4   "done": false,
5   "priority": "alta"
6 }
```

Body

201 Created • 64 ms • 364 B • Save Response

{ } JSON Preview Visualize

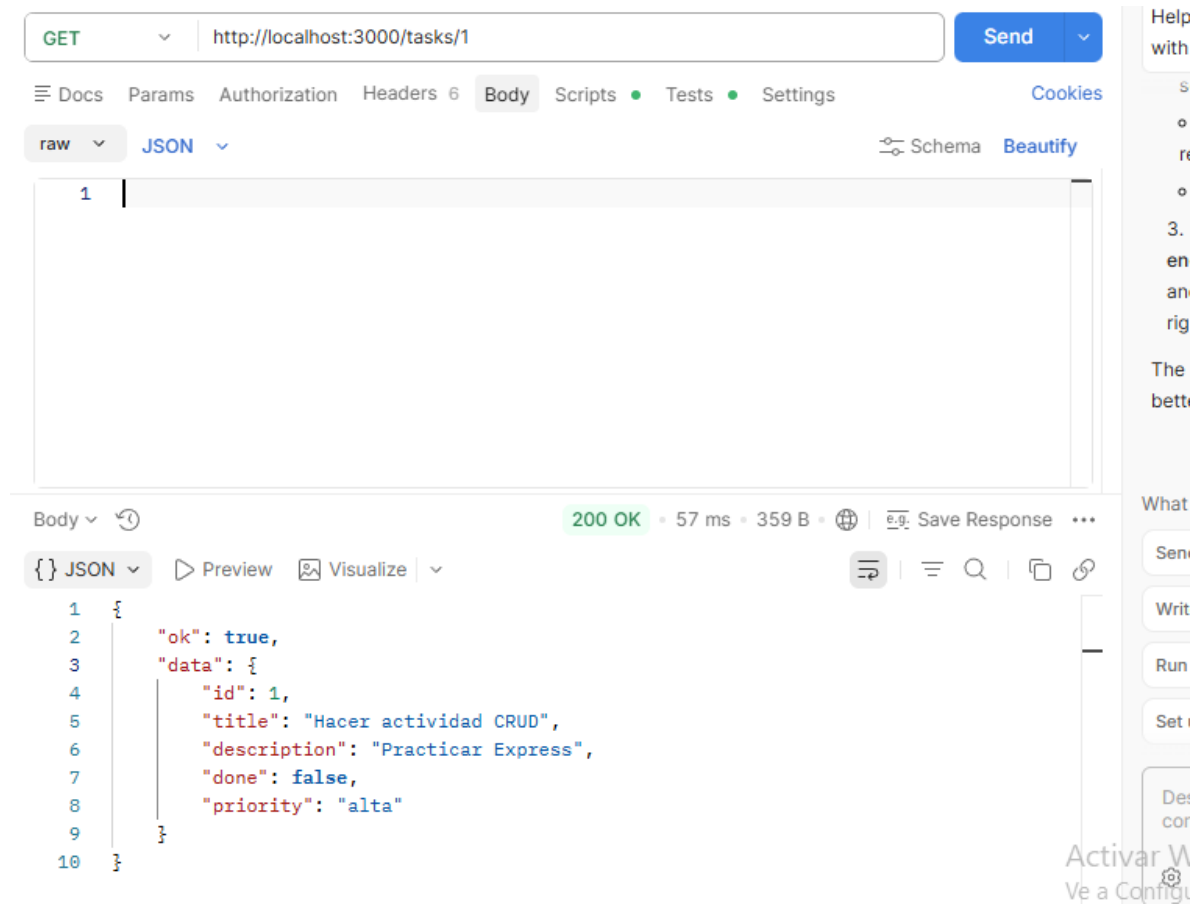
Copy

```
1 {
2   "ok": true,
3   "data": {
4     "id": 1,
5     "title": "Hacer actividad CRUD",
6     "description": "Practicar Express",
7     "done": false,
8     "priority": "alta"
9   }
10 }
```

Activar
Ve a Con

Método get por ID

Se realizó una consulta mediante el endpoint GET /tasks/:id para obtener una tarea específica utilizando su identificador único. La respuesta fue exitosa con código HTTP 200.



Método: PUT para actualizar una tarea

Se utilizó el método PUT para modificar la información de una tarea existente. Los cambios fueron procesados correctamente y la API devolvió código HTTP 200.

My Collection > Get data

PUT http://localhost:3000/tasks/1

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "title": "Actividad terminada",
3   "description": "CRUD completado",
4   "done": true,
5   "priority": "media"
6 }
```

Body Cookies Headers 7 Test Results 1/1 200 OK • 13 ms • 3

{ } JSON Preview Visualize

```
1 {
2   "ok": true,
3   "data": {
4     "id": 1,
5     "title": "Actividad terminada",
6     "description": "CRUD completado",
7     "done": true,
8     "priority": "media"
9   }
10 }
```

Activar Win...
Ve a Configurac...

Prueba del método DELETE para eliminar una tarea

Se ejecutó el método DELETE para eliminar una tarea mediante su identificador. La operación fue completada correctamente con código HTTP 204.

My Collection > Get data Save Share

DELETE http://localhost:3000/tasks/1 Send

Body Cookies

raw JSON Schema Beautify

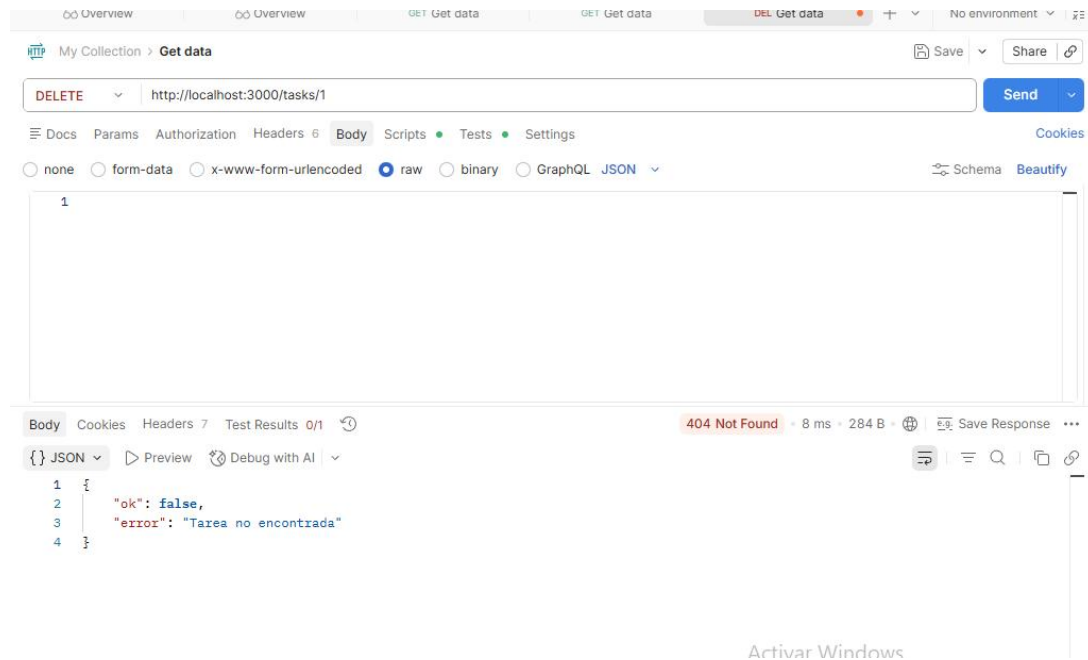
```
1 {
2   "title": "Nueva tarea",
3   "description": "Eliminar tarea",
4   "done": false,
5   "priority": "alta"
6 }
```

Body 204 No Content • 45 ms • 134 B • e.g. ...

Raw 1

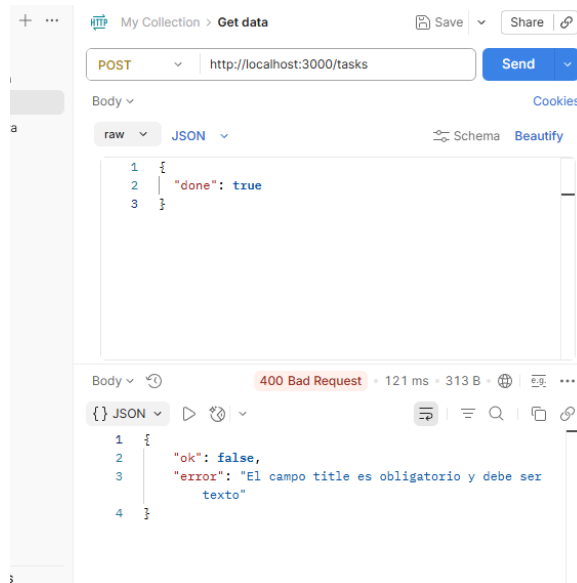
Prueba de recurso no encontrado error 404

Se realizó una consulta a un identificador inexistente para validar el manejo de errores de la API. El sistema respondió correctamente con código HTTP 404 y un mensaje indicando que la tarea no fue encontrada.



Validación de datos incorrectos error 400

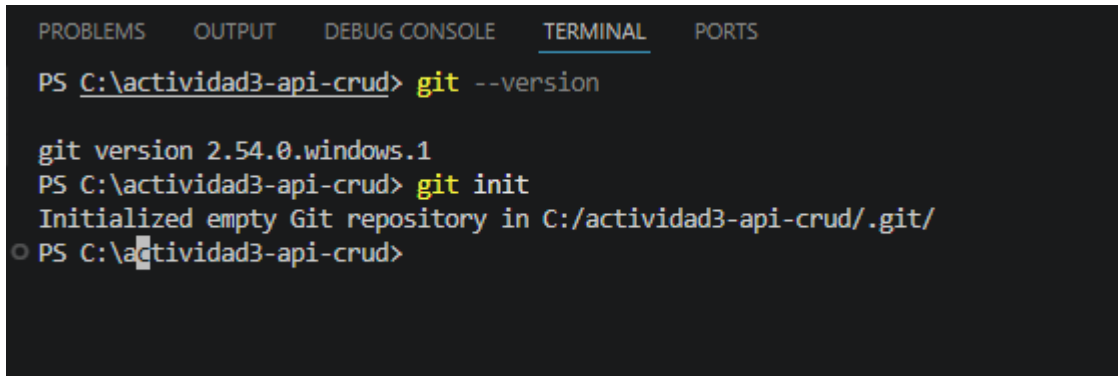
Se envió una petición POST con datos incompletos para comprobar la validación implementada mediante middleware. La API respondió correctamente con código HTTP 400 indicando un error en los datos enviados.



Verificar Git

Se verificó la instalación de Git mediante el comando `git --version`, confirmando que la herramienta se encontraba correctamente instalada para el control de versiones y publicación del proyecto en GitHub.

Se inicializó un repositorio local utilizando Git para llevar el control de versiones del proyecto y preparar la conexión con GitHub.

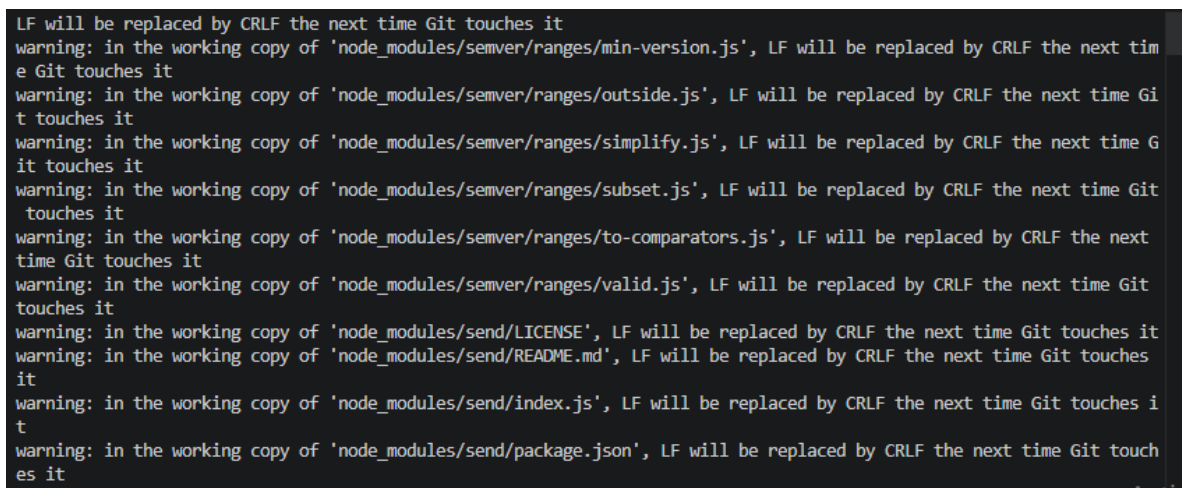


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\actividad3-api-crud> git --version

git version 2.54.0.windows.1
PS C:\actividad3-api-crud> git init
Initialized empty Git repository in C:/actividad3-api-crud/.git/
PS C:\actividad3-api-crud>
```

Agregar archivos



```
LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/semver/ranges/min-version.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/semver/ranges/outside.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/semver/ranges/simplify.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/semver/ranges/subset.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/semver/ranges/to-comparators.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/semver/ranges/valid.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/send/LICENSE', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/send/README.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/send/index.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'node_modules/send/package.json', LF will be replaced by CRLF the next time Git touches it
```

Crear commit

`git commit -m "Actividad 3 API RESTful CRUD"`

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

inspect/test/circular.js
create mode 100644 node_modules/object-inspect/test/deep.js
create mode 100644 node_modules/object-inspect/test/element.js
create mode 100644 node_modules/object-inspect/test/err.js
create mode 100644 node_modules/object-inspect/test/fakes.js
create mode 100644 node_modules/object-inspect/test/fn.js
create mode 100644 node_modules/object-inspect/test/global.js
create mode 100644 node_modules/object-inspect/test/has.js
create mode 100644 node_modules/object-inspect/test/holes.js
create mode 100644 node_modules/object-inspect/test/indent-option.js
create mode 100644 node_modules/object-inspect/test/inspect.js
create mode 100644 node_modules/object-inspect/test/lowbyte.js
create mode 100644 node_modules/object-inspect/test/number.js
create mode 100644 node_modules/object-inspect/test/quoteStyle.js
create mode 100644 node_modules/object-inspect/test/toStringTag.js
create mode 100644 node_modules/object-inspect/test/undef.js
create mode 100644 node_modules/object-inspect/test/values.js
create mode 100644 node_modules/object-inspect/util.inspect.js
create mode 100644 node_modules/on-finished/HISTORY.md
create mode 100644 node_modules/on-finished/LICENSE
create mode 100644 node_modules/on-finished/README.md
create mode 100644 node_modules/on-finished/index.js
create mode 100644 node_modules/on-finished/package.json
```

Crear repositorio en GitHub

 Nuevo repositorio

Escribe para buscar

Crear un nuevo repositorio

Los repositorios contienen los archivos y el historial de versiones de un proyecto. ¿Tienes un proyecto en otro sitio? [Importa un repositorio.](#)

Los campos obligatorios están marcados con un asterisco (*).

1

General

Propietario *

yoselinolis0104-maker

Nombre del repositorio *

actividad3-api-crud

Actividad3-API-CRUD está disponible.

Los nombres de los grandes repositorios son cortos y memorables. ¿Qué tal si [Fiesta jubilante?](#)

Descripción

0 / 350 caracteres

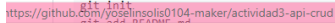
2

Configuración

Elige visibilidad *

Elige quién puede ver y compromete en este repositorio

Público



Se realizó la publicación del proyecto en GitHub utilizando comandos de Git para agregar archivos, crear commits y subir el repositorio remoto.

```
PS C:\actividad3-api-crud> git init
Reinitialized existing Git repository in C:/actividad3-api-crud/.git/
PS C:\actividad3-api-crud> git add .
PS C:\actividad3-api-crud> git commit -m "Actividad 3 API RESTful CRUD"
On branch master
nothing to commit, working tree clean
PS C:\actividad3-api-crud> git branch -M main
PS C:\actividad3-api-crud> git remote add origin https://github.com/yoselinsolis0104-maker/actividad3-api-crud.git
PS C:\actividad3-api-crud> git push -u origin main
Enumerating objects: 1051, done.
Counting objects: Counting objects: Counting objects: Counting objects: Counting objects: Counting objects: Counting objects:
Counting objects: Counting objects: Counting objects: Counting objects: Counting objects: Counting objects: Counting objec
ts: 1Counting objects: 1Counting objects: 1Counting objects: 1Counting objects: 1Counting objects: 1Counting ob
jects: 1Counting objects: 1Counting objects: 1Counting objects: 2Counting objects: 2Counting objects: 2Counting
objects: 2Counting objects: 2Counting objects: 2Counting objects: 2Counting objects: 2Counting objects: 2Count
ing objects: 2Counting objects: 3Counting objects: 3Counting objects: 3Counting objects: 3Counting objects: 3Co
unting objects: 3Counting objects: 3Counting objects: 3Counting objects: 3Counting objects: 3Counting objects:
4Counting objects: 4Counting objects: 4Counting objects: 4Counting objects: 4Counting objects: 4Counting objects
: 4Counting objects: 4Counting objects: 4Counting objects: 4Counting objects: 5Counting objects: 5Counting obje
cts: 5Counting objects: 5Counting objects: 5Counting objects: 5Counting objects: 5Counting objects: 5Counting o
bjects: 5Counting objects: 5Counting objects: 6Counting objects: 6Counting objects: 6Counting objects: 6Countin
```

Publicación del proyecto en GitHub

Finalmente, el proyecto fue publicado en GitHub utilizando Git para el control de versiones. Se verificó correctamente la creación y sincronización del repositorio remoto.

The screenshot shows the GitHub interface for a repository named 'actividad3-api-crud' by user 'yoselinsolis0104-maker'. The repository is public and has 0 stars and 0 forks. The main content area displays a commit history table with 3 commits. The most recent commit, 7b257e5, is titled 'Eliminar node_modules' and was made 34 minutes ago. The commit details show a file named 'src' being deleted. Below the commit history, the README file is visible, titled 'API RESTful CRUD con Express'. The right sidebar contains the 'About' section, which states 'No description, website, or topics provided.', and the 'Lanzamientos' (Releases) section, which indicates 'No se publicaron publicaciones' (No publications were published).

Commit ID	Commit Message	Time	Commits
7b257e5	Eliminar node_modules	hace 34 minutos	3 Commits
	src	Eliminar node_modules	hace 38 minutos
	.gitignore	Eliminar node_modules	hace 34 minutos
	README.md	Eliminar node_modules	hace 38 minutos
	package-lock.json	Actividad 3 API RESTful CRUD	hace 1 hora
	package.json	Actividad 3 API RESTful CRUD	hace 1 hora

API RESTful CRUD con Express

Conclusión

La realización de esta actividad permitió comprender el funcionamiento de una API RESTful mediante la implementación de operaciones CRUD utilizando Node.js y Express. Durante el desarrollo se aplicaron conceptos importantes como la organización del proyecto por responsabilidades, el uso de rutas, controladores y middlewares, así como el manejo adecuado de respuestas HTTP y validaciones.

Además, se reforzó el uso de herramientas como Postman para realizar pruebas de funcionamiento y GitHub para el control de versiones y publicación del proyecto. Gracias a esta práctica se logró desarrollar una API funcional y organizada, mejorando los conocimientos sobre el desarrollo backend y la comunicación entre cliente y servidor.